A Distilled 4.8M-Parameter ResNet for CIFAR-10

Mirsaid Abbasov (ma9197), solo team

NYU Tandon School of Engineering, Deep Learning (Fall 2026), Project 1
Code: https://github.com/ma9197/dl-project1-resnet
Kaggle team: Abbasov

Abstract

I designed and trained, from scratch, a residual network for CIFAR-10 with at most 5 million trainable parameters. The final model has three residual layers with 84, 168 and 336 channels and two basic blocks per layer (4,781,374 parameters). The recipe has three parts: a layout that spends the parameter budget on width at high resolution, strong data augmentation (crop, flip, Cutout, TrivialAugment and CutMix) with a long cosine schedule, and knowledge distillation from larger teachers that are used only during training. The plain model reaches 96.73% on the CIFAR-10 test set (97.00% when averaging an image and its mirror image); the distilled model with the same architecture reaches 97.82% (97.89%). On the competition's held-out test set it scores 0.907 on the public leaderboard, up from 0.884 for the plain model, and my final entry scores 0.874 on the hidden half (rank 28 of 81). Main findings: (1) three wide residual layers beat ResNet-18 style four-layer layouts at this budget; (2) the training recipe mattered more than the layout; (3) distillation with long training added about 0.9 points on CIFAR-10 at no inference cost; (4) on the competition images, which come from a shifted distribution, more CutMix raised the score more than its CIFAR-10 accuracy suggested; (5) test-time adjustments that looked harmless on CIFAR-10 lowered the competition score, so I did not use them.

1 Introduction

A residual network (ResNet) (He et al. 2016) is a convolutional network built from residual blocks that compute $\mathrm{ReLU}(S(x) + F(x))$, where F is conv $\to$ BN $\to$ ReLU $\to$ conv $\to$ BN (Ioffe and Szegedy 2015) and S is the skip connection. The task is to find the most accurate modified ResNet on a held-out version of CIFAR-10 (Krizhevsky 2009) (2,000 images with hidden labels, scored by accuracy on Kaggle), with two rules: at most 5,000,000 trainable parameters, and no pretrained weights or extra data. The design variables are the number of residual layers N, and for layer i the number of blocks B_i, channels C_i, conv kernel size F_i and skip kernel size K_i, plus the average-pool size P. ResNet-18 itself has 11.2M parameters, so it has to be cut down.

I trained on the 50,000 standard CIFAR-10 training images. I checked that the competition's training images are exactly this set (same order, pixels and labels). All code, logs and the final model are in the GitHub repository, and `results.ipynb` shows the executed results, including the parameter count and final test accuracy.

Run	Channels	Blocks	Params	Val	Test
A	64-128-256	3-3-3	4.33M	92.26	91.68
B	**84-168-336**	**2-2-2**	**4.78M**	**92.92**	**92.39**
C	64-128-256-512	1-1-1-1	4.90M	90.48	89.98
D	64-128-256-320	2-2-2-1	4.52M	91.72	91.86

Table 1: Pilot runs (15 epochs, single seed). Val: 5,000 held-out training images. Test: CIFAR-10 test set, accuracy in %.

2 Methodology

2.1 Where the Parameters Go

A basic block with C input and output channels and $F \times F$ kernels has $2F^2C^2$ weights (plus $4C$ for batch norm), so doubling the width makes every block four times more expensive. In ResNet-18 (64-128-256-512 channels, two blocks per layer) about 75% of the parameters sit in the last layer, which works on a 4×4 feature map. To get under 5M there are three options: make all layers narrower, use fewer blocks, or drop the fourth layer and spend the saved parameters on width in the first three. The compute, by contrast, is spread evenly: halving the resolution while doubling the channels keeps the cost of each conv the same.

2.2 Pilot Study: Choosing the Layout

I compared four layouts close to the limit with a short, identical recipe (15 epochs). For these runs I trained on the first 45,000 training images and used the last 5,000 as a validation set to make the choice (Table 1).

Layout B, three wide layers with two blocks each, was best on both metrics. Layout C is ResNet-18 with one block per layer: it has the most parameters but the fewest layers, with most of its capacity at 4×4, and came last. Layout D adds a narrow fourth layer, which also lost to putting those parameters into width. Layout A is deeper but narrower and lost 0.7 points. Gaps under about 0.5 points could be noise, but B led on both metrics. The pros and cons I saw:

- **Three wide layers (B):** more channels at 32×32 and 16×16, where most of the detail of a tiny image is; the cost is more compute (about 0.72 GMACs per image) and a smaller receptive field.
- **Four layers (C, D):** the standard shape with a larger receptive field, but under 5M the 4×4 stage takes most

Run	Epochs	TrivialAug.	Test	+ TTA
e50_B_base	50	no	95.41	95.68
e50_B_ta	50	yes	94.80	95.18
final_B_base_e200	200	no	96.52	96.83
final_B_ta_e200	200	yes	**96.73**	**97.00**

Table 2: TrivialAugment at 50 and 200 epochs (layout B, no distillation). The 200-epoch runs use all 50,000 training images.

of the parameters and the early layers must stay narrow.

- **Deeper and narrower (A):** more nonlinear layers per parameter, but less capacity per layer.
- **Kernel sizes:** I kept $F = 3$, since a 5×5 kernel costs $25/9 \approx 2.8\times$ more per conv. The skip is the identity when the shape is unchanged and a 1×1 conv + BN ($K = 1$) otherwise, the cheapest option. I did not test these two choices by experiment.
- **Pool size:** global average pooling over the final 8×8 map ($P = 8$) has no parameters; a smaller P would only enlarge the final linear layer.

2.3 Training Recipe

- **Optimizer:** SGD with Nesterov momentum 0.9 (Sutskever et al. 2013), batch 128, peak learning rate 0.1 with 2 warmup epochs (Goyal et al. 2017) and cosine decay to 0 (Loshchilov and Hutter 2017).
- **Regularization:** weight decay 5×10^{-4} on conv and linear weights only, and label smoothing 0.1 (Szegedy et al. 2016).
- **Augmentation:** random crop (4-pixel padding), horizontal flip and a 16×16 Cutout (DeVries and Taylor 2017); optionally TrivialAugmentWide (Müller and Hutter 2021) before these.
- **Speed:** the dataset stays on the GPU and crop, flip and Cutout run there per batch, with bfloat16 mixed precision and the channels-last layout in PyTorch (Paszke et al. 2019). A 15-epoch pilot takes 2 minutes on the RTX 4090 Laptop GPU I used.
- **Test-time augmentation (TTA):** I average the predictions for an image and its horizontal flip.

2.4 Training Length and Augmentation

With layout B fixed, I tested TrivialAugment at two lengths (Table 2). At 50 epochs it cost 0.5 points; at 200 epochs it gained 0.2. Figure 1 shows why: with TrivialAugment the training loss stays much higher, and the model reaches only 91.3% accuracy on its augmented training batches, against 99.5% without it. A short schedule turns this extra difficulty into underfitting; with a long one, the model without it nearly memorizes its data, while the model with it keeps improving and passes the baseline in the last 25 epochs.

2.5 Knowledge Distillation

The parameter limit applies to the submitted model only, and the project allows teacher-student distillation, so I trained larger teachers and distilled them into layout B (Hinton, Vinyals, and Dean 2015). For a batch of augmented images x, the student s minimizes

$$\mathcal{L} = \alpha\,\mathrm{CE}(y, s(x)) + (1-\alpha)\,T^2\,\mathrm{KL}\big(\bar{p}_T(x) \,\|\, \sigma(s(x)/T)\big),$$

where $\bar{p}_T(x)$ is the average of the teachers' softmax outputs at temperature T and σ is the softmax. Following Beyer et al. (2022), the teachers see exactly the same augmented image as the student ("consistent" teaching), so the target changes with every crop, Cutout and TrivialAugment operation, and the student is trained for a long time ("patient" teaching). I used $\alpha = 0.1$ and $T = 1$, because my teachers were trained with label smoothing, and Chandrasegaran et al. (2022) found such teachers distill well at low temperature but poorly at high temperature. The student uses the same recipe as before (TrivialAugment, label smoothing on the hard-label term).

Tables 3 and 4 and Figure 2 show the result. With four B-sized teachers the student reached 97.27% (TTA), close to the 97.31% of the ensemble of those teachers. Replacing two of them with wider networks raised the student to 97.58% after 500 epochs and 97.69% after 800. Figure 2 plots accuracy against the fraction of the schedule: the distilled runs are ahead of plain training throughout.

Teachers (runs)	Params	Ep.	Test	+ TTA
B (4)	4.78M	200	96.5 to 96.8	96.8 to 97.1
Wide (2)	11.09M	300	97.41, 97.45	97.61, 97.66
Big (2)	36.69M	300	97.66, 97.67	97.86, 97.92
Big + CutMix (2)	36.69M	400	97.66, 97.85	97.83, 97.99

Table 3: Teachers, used only during training and never submitted. Wide: channels 128-256-512, two blocks per layer. Big: channels 160-320-640, four blocks per layer.

Run	Teachers	CM	Ep.	Test	TTA	Pub.	Priv.
plain	none	0	200	96.73	97.00	.884	.852
kd	4B	0	300	97.10	97.27	.885	.863
kd2	2W+2B	0	500	97.51	97.58		
kd3	2W+2B	0	800	97.57	97.69	.887	.867
kd4	2L+2W	0	800	97.52	97.68		
kd5	2L+2W	0.5	800	97.61	97.73	.895	.867
kd7	2L+2W	0.5	1200	97.76	97.86	.890	.872
kd6	2L+2W	1	1200	97.69	97.77	.903	.871
kd8	2C+2W	1	1200	97.67	97.72	.907	.874
kd10	**2C+2W**	**1**	**1200**	**97.82**	**97.89**	**.907**	**.871**

Table 4: All students use layout B (4.78M parameters). Teachers: B, W(ide), L (big), C (big + CutMix). CM: probability of CutMix per batch. Test and TTA: CIFAR-10 accuracy in %. Pub. and Priv.: Kaggle score on the public and the hidden half, blank if not uploaded. kd8 and kd10 differ only in seed.

2.6 Bigger Teachers and CutMix

Better teachers alone stopped helping. Big teachers reach about 97.9% (TTA), but the student kd4 stayed at 97.68%, the same as kd3. I then added CutMix (Yun et al. 2019): with probability CM, a random rectangle of each image is replaced

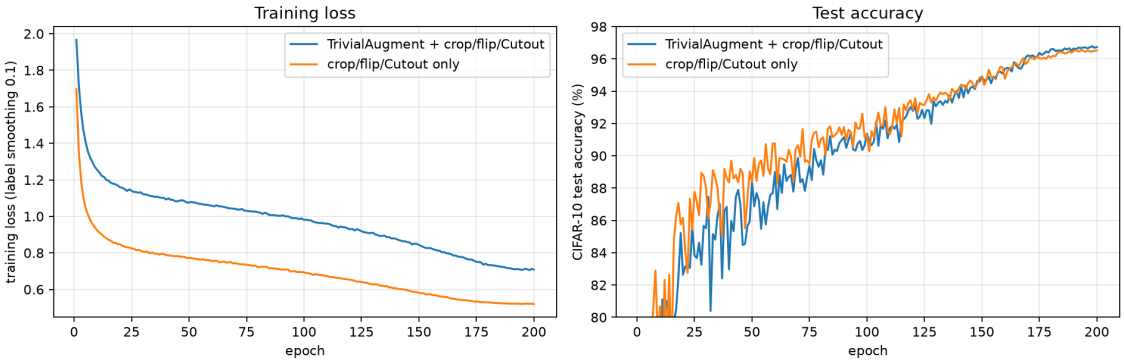


Figure 1: 200-epoch training with and without TrivialAugment. Left: training loss (with label smoothing). Right: CIFAR-10 test accuracy after each epoch (no TTA).

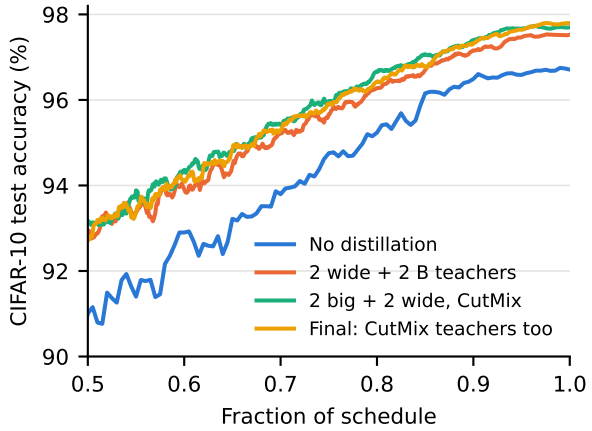


Figure 2: CIFAR-10 test accuracy (no TTA) over the second half of training, against the fraction of each run's schedule, averaged over 1% of the schedule.

by the same rectangle from another image in the batch. The hard-label loss is weighted by the area of each image, and the teachers see the mixed image, so the distillation target is simply what they predict for it.

On CIFAR-10 this gave small gains that are close to the difference between seeds. On the competition images it did more: the public score was 0.887 without it (kd3), 0.895 at CM $= 0.5$ with the bigger teachers (kd5), and 0.903 at CM $= 1$ (kd6). Training the big teachers with CutMix as well raised it to 0.907 (kd8 and kd10). Since each public score has noise (see below), I read this as a trend and not as an exact ranking of the runs. On the hidden half, shown after the competition, the same steps gave a smaller gain (0.867, 0.867, 0.871, 0.874).

2.7 Compute

The pilots, the 50- and 200-epoch runs, and the first distillation round ran on one RTX 4090 Laptop GPU (32 minutes for 200 epochs). All later runs used A100 GPUs on NYU HPC (Cloud Bursting): about 30 minutes for a wide teacher, 73 to 98 minutes for a big teacher, and about 5 hours for a 1200-epoch student. A further student (four big teachers) was cancelled by the cluster at epochs 973 and 1000 of 1200, so it has no result. On HPC the standard CIFAR-10 download was blocked, so I used the copy on Hugging Face (`uoft-cs/cifar10`); I verified that its test set is byte-identical and that its training set has the same 50,000 images and labels (in a different order, which does not matter here).

Part	Output	Details	Params
Stem	84×32^2	conv 3×3, BN	2,436
Layer 1	84×32^2	2 blocks	254,688
Layer 2	168×16^2	2 blocks	904,848
Layer 3	336×8^2	2 blocks	3,616,032
Head	10	pool, linear	3,370
Total			**4,781,374**

Table 5: Trainable parameters of the final model (checked with `torchsummary`; limit 5,000,000). Layer 3 holds 75.6% of them.

3 Results

3.1 Final Model

The final model is layout B distilled for 1200 epochs from two CutMix-trained big teachers and two wide teachers, with CutMix on every batch (`kd10_B_e1200_cmT_s3`). In the project's notation: $N = 3$, $B = (2, 2, 2)$, $C = (84, 168, 336)$, $F = (3, 3, 3)$, $K = (1, 1, 1)$ where the shape changes, and $P = 8$ (Figure 3, Table 5).

3.2 CIFAR-10 Test Accuracy

The final model reaches **97.82%** on the 10,000 CIFAR-10 test images and **97.89%** with flip TTA (Table 6). Most errors are between cats and dogs (37 cats predicted as dogs and 30 dogs as cats, down from 51 and 34 without distillation), followed by trucks predicted as cars (11) and airplanes predicted as ships (10, down from 19).

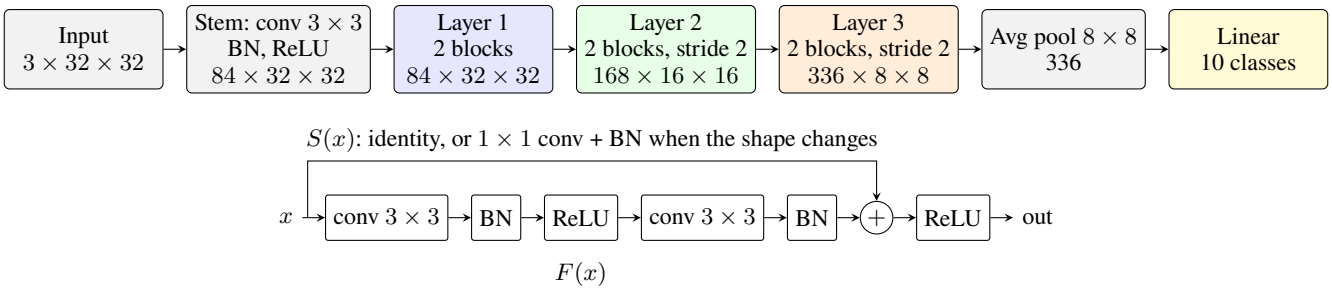


Figure 3: Final architecture (top) and the basic block (bottom). The first block of layers 2 and 3 uses stride 2 and a 1×1 projection on the skip path; all other skips are the identity.

plane	car	bird	cat	deer	dog	frog	horse	ship	truck
98.4	99.0	97.5	94.7	98.6	96.2	99.2	98.5	98.9	98.1

Table 6: Per-class test accuracy (%) of the final model with flip TTA, averaging softmax outputs as the Kaggle predictions do (97.9% overall).

3.3 Competition Test Set

The competition's 2,000 test images are a re-collected, CIFAR-like set rather than the standard test set: they look like natural CIFAR images (no added noise or compression), but they are slightly brighter (mean pixel 125.1 against 121.5) and the model is less confident on them (mean top-class probability 0.78 against 0.83). Such re-collected test sets are known to lower accuracy by several points (Recht et al. 2019). The public score uses half of the images (about 1,000), so it has a standard error of roughly 1 point; each image is worth 0.001. Table 4 lists the main submissions, and Kaggle lets me pick two for the final ranking: kd10 and kd8. The final score is the better of the two on the hidden half: 0.874 (kd8), rank 28 of 81 teams.

CIFAR-10 accuracy was a weak guide to this score. kd7 beat kd6 on CIFAR-10 (97.86% against 97.77%) but scored 1.3 points lower, and kd8 and kd10 tie at 0.907 while differing on 92 of the 2,000 predictions. The final model changes 7.0% of the competition predictions compared with the plain model, against 2.1% of the CIFAR-10 predictions, so the two test sets behave differently.

The hidden half was harder than the public half (0.86 to 0.88 against 0.88 to 0.91), and it reordered close submissions: the class-balanced predictions, worst on the public half among my late uploads, were best on the hidden half (0.879), and the shifted kd8 beat plain kd8 there. Differences of a few images on either half are noise. Over the whole project, distillation still raised the hidden-half score from 0.852 to 0.871 to 0.874.

3.4 Test-Time Ideas I Rejected

The professor allowed test-time BN adaptation (Schneider et al. 2020) and class-balanced prediction on the course Slack, provided they are disclosed. I tested both, using the unlabeled test images but no retraining, and the final predictions use neither (`test_time_checks.py`).

Prediction	CIFAR-10	Public	Private
kd10, flip TTA (final)	**97.91**	**0.907**	**0.871**
kd10 + 5 shifts	97.94	0.903	0.873
kd10 + 9 shifts	97.95	0.902	0.867
kd8, flip TTA (other pick)	97.71	0.907	0.874
kd8 + 9 shifts	97.82	0.902	0.878
kd10 + class balancing	see text	0.895	0.879

Table 7: Test-time ideas I tried and did not use. Shifts: also averaging 1-pixel shifts of the image. CIFAR-10 accuracy here averages softmax outputs, as the Kaggle predictions do. Private: the hidden half, shown only after the competition closed. Final leaderboard: rank 28 of 81 teams, 0.874 (kd8, the better of my two picks on the hidden half).

- **BN statistics from the test images.** On random 2,000-image CIFAR-10 test subsets, replacing the BN statistics with those of the subset lowered accuracy from 97.89% to 96.32%, probably because the training statistics come from heavily augmented images. Mixing 20% of them into the training statistics gave 97.88%. I did not upload this.
- **Class balancing.** I shifted the class scores until each class was predicted about equally often. On CIFAR-10 subsets this gave 98.21% against 98.02% when the subset had equal classes, but 96.20% against 97.90% on random subsets. On Kaggle it changed 66 predictions and scored 0.895, which is 12 images worse on the public half.
- **Shift averaging** (Table 7). It gained up to 0.11 points on CIFAR-10 but lost 4 or 5 images on the public half, on each model I uploaded. On the hidden half the results were mixed (Table 7), so I cannot call either idea better or worse than plain flip TTA.

4 Lessons Learned

- **Spend the budget on width where the resolution is high.** A fourth layer at 4×4 was a poor use of parameters compared with wider early layers.
- **The recipe matters more than the layout.** The same network went from 92.4% (15 epochs) to 97.8% (long training and distillation), while the four layouts were within

2.4 points of each other.

- **Judge augmentation at the final training length.** From the 50-epoch runs alone I would have dropped TrivialAugment, which was the better choice at 200 epochs.
- **Distillation is a free lunch at inference time, up to a point.** Teachers can be as large as the training budget allows, since only the student is submitted. Going from 11M to 37M teachers no longer helped by itself; CutMix and longer schedules did.
- **CIFAR-10 is a weak guide on a shifted test set.** Regularization that changed little on CIFAR-10 moved the competition score the most, and test-time ideas that looked safe on CIFAR-10 lowered the public score.
- **Half a test set is a noisy judge.** With about 1,000 images per half, choices made on the public score did not all hold on the hidden half; only large, consistent gains (here, distillation) did.
- **A fast pipeline pays off.** Two-minute pilots made it cheap to test ideas before long runs.
- **Limitations.** I used the CIFAR-10 test set to compare the final candidates, so those numbers may be slightly optimistic. I also chose among 12 uploads using the public score, and the hidden half confirmed that 0.907 was optimistic: my final entry scored 0.874 there. Two seeds of the same recipe gave 97.72% and 97.89%, so smaller gaps on CIFAR-10 mean little. I did not tune the distillation settings (α, T).

References

Beyer, L.; Zhai, X.; Royer, A.; Markeeva, L.; Anil, R.; and Kolesnikov, A. 2022. Knowledge Distillation: A Good Teacher Is Patient and Consistent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Chandrasegaran, K.; Tran, N.-T.; Zhao, Y.; and Cheung, N.-M. 2022. Revisiting Label Smoothing and Knowledge Distillation Compatibility: What was Missing? In *Proceedings of the 39th International Conference on Machine Learning (ICML)*.

DeVries, T.; and Taylor, G. W. 2017. Improved Regularization of Convolutional Neural Networks with Cutout. *arXiv preprint arXiv:1708.04552*.

Goyal, P.; Dollár, P.; Girshick, R.; Noordhuis, P.; Wesolowski, L.; Kyrola, A.; Tulloch, A.; Jia, Y.; and He, K. 2017. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. *arXiv preprint arXiv:1706.02677*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Ioffe, S.; and Szegedy, C. 2015. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.

Krizhevsky, A. 2009. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto.

Loshchilov, I.; and Hutter, F. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. In *International Conference on Learning Representations (ICLR)*.

Müller, S. G.; and Hutter, F. 2021. TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Paszke, A.; et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Recht, B.; Roelofs, R.; Schmidt, L.; and Shankar, V. 2019. Do ImageNet Classifiers Generalize to ImageNet? In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

Schneider, S.; Rusak, E.; Eck, L.; Bringmann, O.; Brendel, W.; and Bethge, M. 2020. Improving Robustness Against Common Corruptions by Covariate Shift Adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Sutskever, I.; Martens, J.; Dahl, G.; and Hinton, G. 2013. On the Importance of Initialization and Momentum in Deep Learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*.

Szegedy, C.; Vanhoucke, V.; Ioffe, S.; Shlens, J.; and Wojna, Z. 2016. Rethinking the Inception Architecture for Computer Vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Yun, S.; Han, D.; Oh, S. J.; Chun, S.; Choe, J.; and Yoo, Y. 2019. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.